
Dual-Layer Agentic Memory with Fast Write Routing and Slow Consolidation

Wenzhi Li
Zhejiang University

Dong Nie
Independent Researcher

Tongtong Lyu
Xiaohongshu

Rui Lan
Xiaohongshu

Peiyao Wang
Independent Researcher

Lingzi Hong
University of North Texas

Weihang Pan*
Zhejiang University

Binbin Lin
Zhejiang University

Boyuan Pan
Xiaohongshu

Yao Hu
Xiaohongshu

Abstract

Large language model (LLM) agents operate in dynamic environments where knowledge continuously evolves. Existing memory systems typically treat external memory as a monotonically growing repository, inevitably leading to retrieval degradation and increasing computational costs over time. We argue that the core challenge is not retrieval alone, but managing the *knowledge lifecycle*: deciding what to externalize, update, or ultimately internalize. Inspired by Complementary Learning Systems (CLS) theory in neuroscience, we propose **Dual-Layer Agentic Memory**, a framework that shifts memory management to the write phase through cost-aware epistemic routing and periodic parametric consolidation. Incoming information is categorized as *non-write*, *write-new*, or *write-update*, and routed through a small-to-large model cascade that minimizes routing overhead while filtering redundant memories. A subsequent write-back phase selectively consolidates high-value external memories into model parameters via supervised fine-tuning. Experiments demonstrate the dual efficiency of our approach: a 1.7B/8B cascade prunes up to 68% of redundant external memory while escalating fewer than 50% of inputs, yet retains over 98% of the downstream QA Exact Match (EM) achieved by an exhaustive retention baseline. We further show that periodic consolidation successfully internalizes external knowledge, allowing the router to adaptively suppress redundant writes as the model’s epistemic boundaries evolve. Overall, our framework presents a unified paradigm for agent memory: selective externalization followed by selective internalization. Code and dataset will be released upon acceptance.

1 Introduction

Large language model (LLM) agents are increasingly deployed in long-horizon, dynamic environments such as multi-session dialogue, autonomous research, personal assistants, and continual decision-making systems. In these settings, knowledge evolves over time rather than remaining static. An agent may repeatedly encounter information that is already captured by its parameters, genuinely novel facts that lie beyond its current parametric memory, or updates that directly conflict with stale internal beliefs. For example, an assistant may accumulate many user interaction logs that are redundant or outdated, while failing to preserve a newly stated user preference or revise a

*Corresponding author. Email: panweihang@zju.edu.cn

previously incorrect assumption. As a result, effective memory management is essential for persistent and coherent reasoning in agent systems.

Most existing approaches tackle this challenge by equipping agents with explicit external memory mechanisms, typically implemented through retrieval-augmented generation (RAG) [1], memory logs [2], hierarchical buffers [3], or structured external memory. These systems improve persistence beyond the context window, but they usually treat external memory as an unconditional log into which observations are monotonically appended [2]. Because filtering mechanisms operate primarily in the *read* phase—relying on downstream queries to condition retrieval—systems cannot proactively reject redundant information at the source. This design leads to a fundamental storage–performance trade-off: as external memory grows, retrieval becomes noisier, more expensive, and more latency-sensitive.

A different line of work attempts to bypass retrieval by directly incorporating new knowledge into model parameters through knowledge editing [4, 5] or continual learning [6]. While this enables fast retrieval-free access, it comes with its own limitations: parameter updates are computationally expensive and prone to interference or catastrophic forgetting. More importantly, both retrieval-heavy and parametric-only approaches implicitly assume a fixed storage substrate. Knowledge is either externalized or internalized, but is rarely treated as something that should be dynamically allocated, revised, and migrated across memory substrates over time.

We argue that this view is fundamentally incomplete. Rather than deferring filtering decisions to a query-driven read phase, the core challenge is managing knowledge across its full lifecycle the moment it enters the system. When a new fact arrives, the agent should immediately evaluate it against its parametric memory to decide whether the fact is redundant, whether it must be written into external memory, and whether it should later be consolidated into model parameters. In particular, external memory should not serve as a permanent dumping ground. Instead, it should function as a selective and temporary store for knowledge that is either missing from parametric memory or in conflict with stale internal beliefs.

This perspective is naturally connected to memory mechanisms in neuroscience. Complementary learning systems (CLS) theory [7] posits that biological memory relies on two interacting subsystems: a fast-learning hippocampus capable of rapidly encoding novel experiences, and a slower neocortex that gradually consolidates stable knowledge into long-term representations, typically via offline replay during sleep. From this perspective, external memory acts as the fast, editable hippocampal buffer for novel or conflict-prone information, while parametric memory serves as the slower, stable cortical substrate. Inspired by this analogy, we formulate agent memory as a *dual-layer knowledge lifecycle* problem consisting of fast write routing and slow consolidation (analogous to sleep-dependent memory replay).

Concretely, we propose a dual-layer system with two types of memory and two forms of decisions. At the **carrier layer**, the agent maintains *parametric memory* for stable, compressed, low-cost knowledge and *external memory* for dynamic, editable, high-risk facts. At the **decision layer**, we separate *fast write routing* from *slow write-back* (the consolidation mechanism). We conceptualize every incoming piece of knowledge through an operational taxonomy of three classes: *non-write*, *write-new*, and *write-update*. Here, *non-write* denotes knowledge already safely answerable from parametric memory, *write-new* denotes knowledge missing from parametric memory, and *write-update* denotes knowledge for which the model produces a non-refusal but incorrect answer, indicating stale or conflicting internal knowledge. To manage admission efficiently, the write router bypasses explicit three-way classification and directly optimizes a continuous storage–performance trade-off through a binary admission decision: *write* or *discard*. Here, *write* and *discard* are router-level actions used for cost-aware decision-making, while the precise handling of novel versus stale facts is deferred to the subsequent consolidation stage.

A central systems contribution of our framework is that write routing is implemented as a *small-to-large cascade*. Instead of relying on a large model for every routing decision, we use a lightweight model to perform cheap front-end screening and resolve easy cases early, while only uncertain or ambiguous knowledge items are escalated to a stronger model. This design is crucial because write routing is itself a resource-allocation problem: if the router always requires the large model, then routing becomes too expensive to justify. The cascade therefore optimizes not only memory usage, but also the computational cost of the routing policy itself.

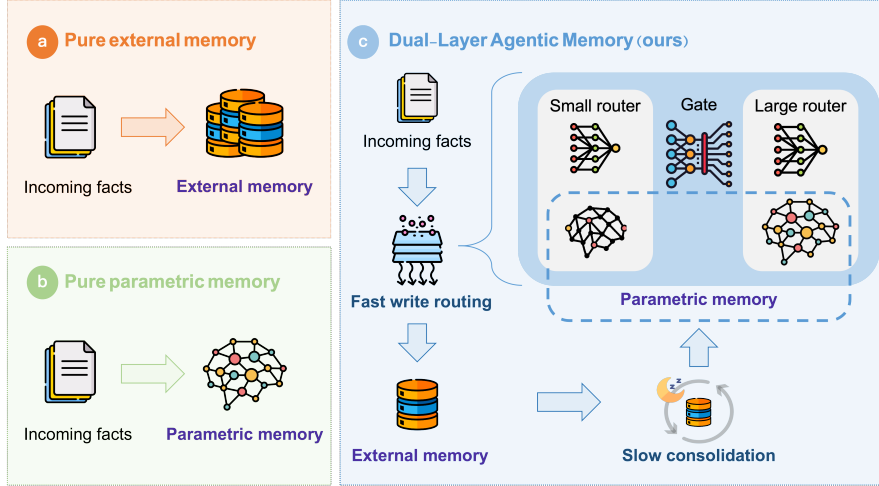


Figure 1: **Comparison of agent memory paradigms.** (a) **Pure external memory:** Monotonically appending facts to an external database often leads to memory bloat and degrades retrieval precision. (b) **Pure parametric memory:** Directly incorporating continuous updates into model weights is computationally expensive and susceptible to catastrophic forgetting. (c) **Dual-Layer Agentic Memory (ours):** We formulate memory as a knowledge lifecycle. A cost-aware cascade router (*Fast Write Routing*) filters incoming facts, escalating uncertain items to maintain a compact external memory. Retained external memories are periodically internalized via supervised fine-tuning (*Slow Consolidation*). This process updates the model’s parametric knowledge, shifting its epistemic boundaries to reduce future reliance on external memory.

To operationalize this formulation, we construct a behavioral labeling pipeline over raw (knowledge, query, answer) triples. For each triple, we compare zero-shot answering against answering with explicit memory support. If the model answers correctly zero-shot, the corresponding knowledge is labeled as *non-write*; if the model fails zero-shot but succeeds with memory, it is write-worthy. We further distinguish *write-new* from *write-update* based on whether the zero-shot failure manifests as an explicit refusal or a confident but incorrect answer. These labels allow us to train and evaluate write routers using model behavior rather than abstract semantic judgments.

Finally, we study *write-back* as the slow half of the lifecycle by converting selected *write-new* and *write-update* memories into fine-tuning supervision and testing whether previously retrieval-dependent facts become internally answerable. In this sense, our framework is not only about selective storage, but also about selective internalization: the agent first externalizes only high-value knowledge, and later consolidates stable, validated memories back into parametric memory to reduce future dependence on retrieval. Because the write router operates directly on the model’s hidden states and uncertainty profiles, it naturally senses this consolidation; once a fact is internalized, its shifted internal representations prompt the router to discard future encounters of the same knowledge, effectively closing the knowledge lifecycle loop.

Our contributions are summarized as follows:

- We formulate agent memory as a **dual-layer knowledge lifecycle system** that unifies external memory, parametric memory, fast write routing, and slow consolidation.
- We introduce an **operational memory taxonomy** (*non-write*, *write-new*, *write-update*) together with a **cost-aware small-to-large routing cascade** that frames memory admission as an accuracy–efficiency optimization problem.
- We propose a periodic write-back consolidation mechanism that enables **knowledge internalization** and adaptive **epistemic shift**, progressively reducing reliance on external memory as parametric knowledge evolves.

- We construct an online **streaming evaluation benchmark** that decouples write-time admission from read-time retrieval, showing that selective externalization and consolidation mitigate memory bloat while closely matching exhaustive-memory performance.

2 Related work

Memory architectures in LLM-based agents. Long-horizon LLM agents typically extend working memory via external RAG [1], with early systems such as Generative Agents and MemGPT introducing long-term memory management [2, 3]. As memory becomes central to lifelong LLM agents [8], recent systems further improve working-memory compression and memory organization through hierarchical chunking, dynamic linking, and structured memory representations [9–11]. Yet these approaches mainly manage active context or organize stored memories, lacking a mechanism to close the knowledge lifecycle loop between external and parametric memory. Consequently, external memory can accumulate redundant facts, leading to memory bloat [12] and degraded access [13] despite structured retrieval mechanisms. In contrast, our Dual-Layer Agentic Memory addresses this by formulating memory as a knowledge lifecycle, where fast write routing and slow consolidation work in tandem to achieve selective externalization followed by selective internalization.

Retrieval-augmented generation and routing. To mitigate the latency and noise of exhaustive retrieval, recent frameworks dynamically route memory operations. However, these routing mechanisms operate primarily in the *read* phase, relying on downstream queries to condition retrieval decisions [14–18]. To address the foundational problem of memory admission—deciding what to externalize before storage occurs—we explicitly shift the focus to the *write* phase. Drawing inspiration from cascaded inference, which is widely used to optimize LLM generation [19, 20], we adapt a cost-aware small-to-large cascade to strictly gate memory admission.

Knowledge editing and continual parametric updating. Techniques like ROME [4] and MEMIT [5] update specific model parameters to correct stale facts, while recent methods extend editing to long-form, diverse-format, or parameter-preserving multi-hop settings [21–23]. However, continuous parametric updates are computationally expensive and highly susceptible to catastrophic forgetting [24, 6]. This motivates using external memory as a fast, editable buffer, with parameter modifications reserved for slower offline consolidation.

Neuroscience-inspired memory and consolidation. Our architectural design is motivated by the CLS framework [25, 7], modeling the interplay between the fast-learning hippocampus and the slow-learning neocortex. Recent AI systems adopt CLS-inspired consolidation to prevent forgetting through structured memory, reconstructive persistence, or hybrid memory circuits [26, 27]. Related continual-learning work further shows that sleep-like unsupervised replay can improve retention under limited or imbalanced data [28]. Motivated by this consolidation view, we translate the sleep phase into periodic supervised fine-tuning (SFT), physically consolidating transient external memory into the model’s parameters.

3 Dual-Layer Agentic Memory

We model agent memory as a *dual-layer knowledge lifecycle* problem. Our framework contains two layers. At the **carrier layer**, the agent maintains two memory substrates: parametric memory Θ , which is stable, compressed, and cheap to access, and external memory E , which is explicit, editable, and suited to dynamic or conflict-prone facts. At the **decision layer**, the system contains a fast *write router*, which decides whether incoming knowledge should enter external memory, and a slow *write-back* mechanism, which selectively internalizes high-value external memories into Θ through supervised fine-tuning.

3.1 Operational memory taxonomy

We define write labels based on model behavior under zero-shot versus memory-supported answering. Given a knowledge item f and its associated factual probes, we distinguish three cases:

- *non-write*: the agent can already answer the associated query correctly from parametric memory alone, so external memory is unnecessary;
- *write-new*: the agent produces a refusal answer in the zero-shot setting, and becomes correct when the relevant knowledge is provided, indicating that the fact is missing from parameters;
- *write-update*: the agent produces a non-refusal but incorrect answer in the zero-shot setting, and becomes correct when the relevant knowledge is provided, indicating stale parametric memory.

3.2 Problem setup

Let f denote an incoming knowledge item. At storage time, the router evaluates f to produce a binary admission decision: *discard* (relying solely on parametric memory Θ) or *write* (appending f to external memory E). The lifecycle is:

$$f \xrightarrow{\text{write routing}} \begin{cases} \text{discard} \\ \text{write} \longrightarrow E \xrightarrow{\text{optional write-back}} \Theta' \end{cases} \quad (1)$$

Subsequently, the items written to E are utilized during the slow write-back stage for parametric consolidation, ultimately updating the model to Θ' .

3.3 Small-to-large cost-aware write routing

Write routing is implemented as a **small-to-large cascade**. Physically, each router consists of a frozen Large Language Model (LLM) backbone—serving as a feature extractor—paired with a trained lightweight Multi-Layer Perceptron (MLP) decision head. A small router M_{small} (utilizing a smaller LLM backbone) first performs cheap front-end screening and resolves easy cases. Only uncertain or ambiguous examples are escalated to a stronger but more computationally expensive large router M_{large} (utilizing a larger LLM backbone).

For each fact f , M_{small} predicts whether the fact should be written to external memory ($a_{\text{small}} \in \{W, D\}$). A gating module then estimates whether escalating is worth the additional compute cost. If escalation is triggered, the final decision becomes $a_{\text{large}} \in \{W, D\}$ determined by M_{large} .

3.3.1 Routing objective

Let λ_s denote the storage penalty and λ_e the escalation penalty. Writing a fact is rewarded only when its marginal task-performance gain outweighs the combined penalty $\lambda_s + \lambda_e$. For a given fact, let $\text{EM}(W)$ denote the downstream QA Exact Match (EM) obtained when the fact is written to external memory, and $\text{EM}(D)$ when discarded. We define the write reward as $r(W) = \text{EM}(W) - \text{EM}(D) - \lambda_s$, and the discard reward as $r(D) = 0$.

3.3.2 Escalation gate

The small router predicts rewards $\hat{r}_{\text{small}}(W)$ and $\hat{r}_{\text{small}}(D)$, and the large router predicts $\hat{r}_{\text{large}}(W)$ and $\hat{r}_{\text{large}}(D)$. We define the expected value of stopping at the small router as $V_{\text{small}} = r(a_{\text{small}})$, and the expected value of escalation as $V_{\text{large}} = r(a_{\text{large}}) - \lambda_e$. The gate is trained to predict the gain of escalation: $g^* = V_{\text{large}} - V_{\text{small}}$. At inference time, the example is escalated if and only if the predicted gain is positive ($g_{\text{pred}} > 0$).

3.3.3 Feature construction and neural parameterization

The decision heads for M_{small} and M_{large} consume semantic and uncertainty features extracted from their respective LLM backbones. Let $\{h_t\}_{t=1}^L$ be the final-layer hidden states produced by the LLM for the tokenized fact f . We compute a global semantic embedding using mean pooling:

$$e = \frac{1}{L} \sum_{t=1}^L h_t. \quad (2)$$

To capture epistemic uncertainty, let $\ell_t = -\log p(x_t | x_{<t})$ be the token-level negative log-likelihood. We compute the mean NLL $\bar{\ell} = \frac{1}{L} \sum_{t=1}^L \ell_t$, and resample the sequence NLL curve into a fixed-length

representation:

$$c = \text{Interp}_{64}([\ell_1, \ell_2, \dots, \ell_L]). \quad (3)$$

The resulting uncertainty vector incorporates the sequence length and the resampled curve:

$$u = [\bar{\ell}; \log(1 + L); c]. \quad (4)$$

Using these features, the MLP head of the small router (z_{small}), the escalation gate (z_g), and the MLP head of the large router (z_{large}) respectively consume:

$$z_{\text{small}} = [e_{\text{small}}], \quad (5)$$

$$z_g = [e_{\text{small}}; u_{\text{small}}; \hat{r}_{\text{small}}(W); \hat{r}_{\text{small}}(D); |\hat{r}_{\text{small}}(W) - \hat{r}_{\text{small}}(D)|], \quad (6)$$

$$z_{\text{large}} = [e_{\text{large}}]. \quad (7)$$

All decision heads are parameterized as MLPs (utilizing LayerNorm, GELU activations, and Dropout) trained to regress reward targets using mean squared error, while the LLM backbones remain frozen. At deployment, the policies trace the Pareto frontier across EM–storage–compute trade-offs. **We defer full stagewise training and deployment protocols to Appendix C.**

3.4 Write-back as slow consolidation

We introduce **write-back**, a slow consolidation stage that internalizes selected external memories into model parameters. Let $E = \{f_i\}_{i=1}^N$ denote the routed external memory. Each retained fact f_i is associated with a set of factual probes $\mathcal{Q}_i = \{(q_{i1}, a_{i1}), \dots, (q_{im_i}, a_{im_i})\}$. Write-back is applied only to facts labeled *write-new* or *write-update*.

Each selected fact is converted into two complementary forms of supervision: independent question-level examples ($x_{ij}^{\text{qa}} = q_{ij}, y_{ij}^{\text{qa}} = a_{ij}$) and sentence-level targets (x_i^{sent}, f_i). We optimize the standard SFT causal language modeling objective over assistant tokens:

$$\mathcal{L}_{\text{SFT}}(\Theta) = - \sum_{k=1}^K \sum_{t \in \mathcal{A}_k} \log p_{\Theta}(y_{k,t} \mid x_k, y_{k,<t}). \quad (8)$$

A fact is removed from external memory only if consolidation is validated, i.e., the updated model answers its probes correctly without retrieval: $\frac{1}{m_i} \sum_{j=1}^{m_i} \text{EM}(\text{Ans}_{\Theta'}(q_{ij}), a_{ij}) \geq \tau_{\text{flush}}$. Otherwise, it remains in external memory. For successfully consolidated facts, this completes the knowledge lifecycle: initial externalization into E , subsequent internalization into Θ' , and final eviction from E .

4 Experiments and results

To evaluate the dual-layer knowledge lifecycle, we construct an online streaming benchmark and compare our framework against various memory admission baselines.

4.1 The streaming knowledge lifecycle benchmark

Existing memory benchmarks typically present background knowledge and downstream queries simultaneously. This static setup conflates two distinct challenges: deciding *whether* to store an incoming statement, and deciding *what* to retrieve. Because the query is already known when facts are processed, most systems bypass write-time gating entirely—either unconditionally memorizing all inputs or relying exclusively on the downstream query to filter information at read time. To explicitly isolate the admission bottleneck, we cast memory management as an *online streaming task*. By requiring the agent to make immediate write-versus-discard decisions as facts arrive sequentially—strictly prior to encountering any associated queries—we effectively decouple memory admission from retrieval.

Offline dataset and behavioral labeling. We derive the fact corpus from the Zero-Shot Relation Extraction (ZsRE) benchmark [29]. To establish targets for memory admission, we probe a frozen qwen3-8B baseline on 6,040,560 fact-related queries. By comparing zero-shot answers against memory-supported answers, we quantify the model’s parametric blind spots. Based on these behavioral signals, we map each fact into an operational taxonomy: *non-write*, *write-new*, or *write-update* (see Appendix A for mathematical formulations and label distributions).

Online benchmark environment and trajectory generation. We evaluate continuous memory management over $E = 300$ streaming episodes from the test split. Each episode contains $T = 500$ temporally interleaved conversational turns with $K = 100$ candidate fact injections and downstream QA interactions. This streaming setup simulates realistic agent environments where knowledge injection and reasoning co-occur, requiring the system to decide at write-time whether each incoming fact should be externalized. The injected facts preserve the natural distribution of the offline taxonomy, including *non-write* distractors, while satisfying causal admissibility (i.e., queries never precede supporting facts). A typical interaction trajectory is shown in Figure 3, and the full synthesis procedure is provided in Appendix A.

Metrics. We evaluate downstream QA quality using Exact Match (EM), token-level F_1 , and refusal rate. Storage decision quality is scored at the knowledge-injection level against the fact-level labels via storage ratio together with Store Precision, Store Recall, and Store F_1 (see Appendix A for specific details on memory admission targets). Finally, system efficiency is reported as routing compute cost.

4.2 Baselines and compared methods

We compare against **Static Policies** (No Store, Full Store, Random Store), **Heuristic Policies** (Heuristic Store, PPL-Conditional Store), and **Supervised Policies** (Logistic Regression, MLP Classifier). For our cascaded methods, we explicitly instantiate M_{small} using qwen3-1.7B as the backbone, and M_{large} using qwen3-8B [30]. We evaluate Write Router, which operates on features extracted from the frozen base models, and Write Router_{SFT}, where both the 1.7B and 8B backbones have undergone the write-back consolidation phase to maintain aligned knowledge distributions. **Detailed implementation and thresholding strategies for all heuristic and supervised baselines are provided in Appendix B.** All experiments are conducted on a single compute node equipped with 4 NVIDIA H20 GPUs.

4.3 Main results: end-to-end online benchmark

We evaluate all methods on the online streaming benchmark. Table 1 reports downstream QA quality together with storage and routing compute efficiency. The overarching question is whether our dual-layer framework can approach the performance of exhaustive storage while simultaneously avoiding external memory bloat and exorbitant routing costs.

Limitations of static and heuristic policies. Under the base model framework, relying purely on parametric memory yields poor performance, whereas exhaustive retention provides a theoretical upper bound. As shown in Table 1, while **Static Policies** like Random Store can partially recover accuracy, they are highly memory-inefficient. **Heuristic Policies** (Heuristic Store and PPL-Conditional Store) improve the storage–performance frontier but remain suboptimal compared to **Supervised Policies** (Logistic Regression and MLP Classifier). This indicates that optimal write decisions require explicit training on model-internal features.

Dual efficiency via cascade routing. Beyond storage footprint, the inference overhead of routing is a critical bottleneck. Explicit probing (Heuristic Store) and full-model feature extraction (**Supervised Policies**) incur substantial computational costs. Our Write Router addresses this through a small-to-large cascade. By deploying qwen3-1.7B as an efficient proxy, it escalates only 39.7% to 49.0% of uncertain queries to the 8B model, reducing the expected inference cost by roughly 30% to 39%. Furthermore, it establishes a strictly superior storage–performance frontier for the base model. It decisively outperforms not only static and heuristic baselines but also full-8B **Supervised Policies** at a fraction of the compute cost. Ultimately, the Write Router tightly tracks the theoretical optimum, retaining over 98.3% of the exhaustive retention upper bound (Full Store) while significantly reducing both memory bloat and routing compute overhead.

Knowledge internalization and epistemic shift. Table 1 (bottom) evaluates Write Router_{SFT}. Periodic SFT facilitates **knowledge internalization**, significantly improving baseline QA performance and inducing an **epistemic shift**. Sensing this updated parametric capacity, the router correctly discards newly internalized facts—a behavior that mathematically manifests as an apparent drop in Store Recall when evaluated against static base-model targets. Together, this adaptive discarding and

Table 1: End-to-end results on the dynamic online benchmark. Our cascaded Write Router improves the storage–performance frontier by reducing both routing compute and external memory usage: it escalates only $\sim 40\text{--}49\%$ of inputs to the 8B model, prunes up to 68% of redundant memory, and still retains over 98.2% of Full Store EM.

Policy	QA EM	Tok. F_1	Ref. Rate	Store ($\downarrow$)	Store Prec.	Store Recall	Store F_1	Routing Cost [†] ($\downarrow$)
<i>Base Model Framework (qwen3-8B)</i>								
No Store	38.02%	0.4039	2.8%	0.0%	–	0.0%	–	0
Random Store ($p = 0.5$)	62.95%	0.6517	1.6%	<u>50.0%</u>	76.63%	50.06%	60.56%	0
Random Store ($p = 0.8$)	78.07%	0.8020	0.7%	80.0%	76.60%	80.07%	78.30%	0
Full Store	87.79%	0.8980	0.1%	100.0%	76.54%	100.0%	86.71%	0
Heuristic Store	73.56%	0.7605	0.6%	59.08%	86.06%	66.43%	74.98%	$2 \times \text{Gen}_{8B}$
PPL-Cond. Store	78.95%	0.8144	0.5%	71.74%	83.58%	78.35%	80.88%	$1 \times \text{Fwd}_{8B}$
Logistic Regression	83.84%	0.8611	<u>0.2%</u>	68.50%	95.68%	85.64%	<u>90.38%</u>	$1 \times \text{Fwd}_{8B}$
MLP Classifier	83.18%	0.8548	0.3%	65.36%	96.87%	82.72%	89.24%	$1 \times \text{Fwd}_{8B}$
Write Router ($\lambda_s = 0.2, \lambda_e = 0.008$)	83.97%	0.8622	0.3%	64.81%	<u>95.76%</u>	81.09%	87.82%	$\text{Fwd}_{1.7B} + 49.0\% \text{Fwd}_{8B}$
Write Router ($\lambda_s = 0.08, \lambda_e = 0.005$)	<u>86.35%</u>	<u>0.8846</u>	<u>0.2%</u>	77.00%	91.06%	<u>91.61%</u>	91.33%	$\text{Fwd}_{1.7B} + 39.7\% \text{Fwd}_{8B}$
<i>SFT Model Framework (Knowledge-adapted qwen3-8B)</i>								
No Store	69.16%	0.7154	0.0%	0.0%	–	0.0%	–	0
Full Store	92.29%	0.9303	0.0%	100.0%	76.54%	100.0%	86.71%	0
Write Router _{SFT} ($\lambda_s = 0.2, \lambda_e = 0.008$)	89.77%	0.9169	0.0%	<u>32.08%</u>	97.85%	41.01%*	57.80%	$\text{Fwd}_{1.7B} + 45.8\% \text{Fwd}_{8B}$
Write Router _{SFT} ($\lambda_s = 0.08, \lambda_e = 0.005$)	<u>90.71%</u>	<u>0.9253</u>	0.0%	47.85%	<u>93.07%</u>	<u>58.19%*</u>	<u>71.61%</u>	$\text{Fwd}_{1.7B} + 43.3\% \text{Fwd}_{8B}$

[†]Routing cost measures inference overhead. Gen_{8B} denotes autoregressive generation with the 8B model, while Fwd_{8B} and $\text{Fwd}_{1.7B}$ denote a single forward pass. The cascaded Write Router filters all inputs with the 1.7B model and escalates only a subset to the 8B model.

*Store Recall for Write Router_{SFT} is evaluated against static base-model targets.

the precise targeting of remaining blind spots (97.85% Store Precision) allow the router to drastically reduce the storage footprint to just 32.08%. Despite this minimal external memory reliance, the agent achieves 90.71% QA EM, retaining over 98.2% of the Full Store upper bound.

4.4 Cascade gating analysis and architectures

To determine the optimal cascade parameterization, we conduct ablations on the held-out routing test split. Semantic features combined with epistemic uncertainty, mean pooling, and MSE optimization yield the strongest Pareto frontier (detailed in Appendix D). Figure 2 evaluates whether the thresholded small-to-large cascade can recover large-router performance at lower computational cost.

As shown in Figure 2, the cascade consistently closes the small–large router gap across storage budgets. Even under strict escalation limits, it closely tracks the large-router upper bound, with negligible EM degradation and substantial gains over the standalone small router (see Appendix D.3). This validates our design: escalating only uncertain inputs preserves near-optimal routing quality while reducing memory-admission inference cost.

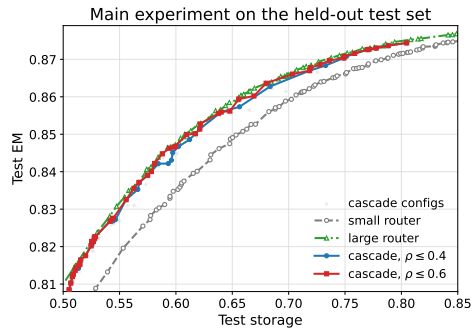


Figure 2: EM–storage Pareto frontiers for the main cascade configuration under escalation thresholds $\rho \in \{0.4, 0.6\}$.

4.5 Write-back and consolidation analysis

The final lifecycle stage, write-back, converts selected external memories into SFT supervision. The objective is *selective internalization*—embedding factual updates into parametric memory (Θ') to reduce subsequent retrieval dependence. As shown previously, the write-back phase alters the agent’s baseline QA performance and reduces the required storage ratio from 64.81% to 32.08%.

Table 2: Post-write-back label transition matrix. While many facts become *non-write*, 1,752 previously stable facts degrade, indicating parametric interference.

Base \ SFT	non-write	write-new	write-update	Total (Base)
non-write	5,347	0	1,752	7,099
write-new	5,785	0	7,939	13,724
write-update	4,898	0	4,482	9,380
Total (SFT)	16,030	0	14,173	30,203

To analyze this epistemic shift at the fact level, we track the behavioral labels of 30,203 held-out knowledge items before and after consolidation in Table 2. The SFT stage alters the labels of 67.46% of the knowledge base, revealing three distinct phenomena:

- **Parametric Internalization:** A total of 10,683 retrieval-dependent facts (5,785 *write-new* and 4,898 *write-update*) transition to *non-write*. This indicates that the model can now answer these queries using parametric memory alone, effectively reducing reliance on external memory.
- **Partial Absorption:** 7,939 *write-new* facts shift to *write-update*. Here, the model moves from explicit zero-shot refusals to generating incorrect answers. This suggests that the model acquires partial semantic familiarity during SFT but still relies on external memory support for exact factual correction.
- **Parametric Interference:** We observe that 1,752 previously stable *non-write* facts degrade into *write-update*. This reflects the catastrophic forgetting common in continuous parametric memory updating. This interference provides empirical motivation for the dual-layer architecture: because slow consolidation can corrupt existing knowledge, maintaining an external memory layer is necessary to override newly induced parametric errors.

5 Discussion & conclusion

In this work, we formalize LLM agent memory not as a passive, monotonically growing repository, but as a dynamic *knowledge lifecycle* problem. Drawing inspiration from complementary learning systems, our dual-layer architecture integrates fast, selective write routing with slow parametric consolidation. Our empirical evaluations yield three primary conclusions. First, **cost-aware selective externalization** via a small-to-large cascaded router efficiently filters redundant facts, preserving downstream accuracy near the theoretical upper bound while drastically reducing both external memory bloat and routing compute overhead. Second, **write-back enables internalization**, leveraging the external memory as a temporary buffer that allows the agent to internalize retrieval-dependent facts through periodic SFT. Because the router is conditioned on the model’s internal states, it adaptively recognizes this shifted parametric capacity and further suppresses the external memory ratio. Finally, **parametric interference motivates the dual-layer design**: our fact-level transition analysis reveals that while continuous parametric updates internalize vast knowledge, they inherently disrupt a subset of previously stable facts. This confirms the necessity of maintaining a fast-routing external memory layer: it acts as an essential buffer to dynamically correct the inevitable epistemic regressions caused by periodic parametric consolidation.

Limitations and future work. While our framework provides a principled memory lifecycle, several limitations motivate future work. First, the offline SFT write-back introduces internalization latency and high compute costs; exploring continuous, parameter-efficient adaptation (e.g., online LoRA) could tighten this loop. Second, treating the external buffer as a uniform dataset during consolidation lacks a principled mechanism to resolve temporally conflicting updates accumulated between cycles. Finally, while the external layer effectively buffers parametric interference, actively

mitigating this catastrophic forgetting during write-back—such as via episodic replay or weight regularization—remains a critical challenge for lifelong agent learning.

Acknowledgments and Disclosure of Funding

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [2] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [4] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022.
- [5] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022.
- [6] Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. Continual learning for large language models: A survey. *arXiv preprint arXiv:2402.01364*, 2024.
- [7] Dharshan Kumaran, Demis Hassabis, and James L McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in cognitive sciences*, 20(7):512–534, 2016.
- [8] Junhao Zheng, Chengming Shi, Xidi Cai, Qiuke Li, Duzhen Zhang, Chenxing Li, Dong Yu, and Qianli Ma. Lifelong learning of large language model based agents: A roadmap. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026.
- [9] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32779–32798, 2025.
- [10] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [11] Ruihong Zeng, Jinyuan Fang, Siwei Liu, and Zaiqiao Meng. On the structural memory of llm agents. *arXiv preprint arXiv:2412.15266*, 2024.
- [12] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. Meminsight: Autonomous memory augmentation for llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33124–33140, 2025.
- [13] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- [14] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, 2024.

- [15] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 conference of the north american chapter of the association for computational linguistics*, 2024.
- [16] Tiziano Labruna, Jon Ander Campos, and Gorka Azkune. When to retrieve: Teaching llms to utilize information retrieval effectively. In *Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing-Natural Language Processing in the Generative AI Era*, pages 623–632, 2025.
- [17] Zijun Yao, Weijian Qi, Liangming Pan, Shulin Cao, Linmei Hu, Liu Weichuan, Lei Hou, and Juanzi Li. Seakr: Self-aware knowledge retrieval for adaptive retrieval augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27022–27043, 2025.
- [18] Di Wu, Jia-Chen Gu, Kai-Wei Chang, and Nanyun Peng. Self-routing rag: Binding selective retrieval with knowledge verbalization. *arXiv preprint arXiv:2504.01018*, 2025.
- [19] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A unified approach to routing and cascading for LLMs. In *International conference on machine learning*, 2025.
- [20] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms from preference data. In *International conference on learning representations*, 2025.
- [21] Houcheng Jiang, Junfeng Fang, Ningyu Zhang, Guojun Ma, Mingyang Wan, Xiang Wang, Xiangnan He, and Tat-seng Chua. Anyedit: Edit any knowledge encoded in language models. *arXiv preprint arXiv:2502.05628*, 2025.
- [22] Changyue Wang, Weihang Su, Qingyao Ai, Yujia Zhou, and Yiqun Liu. Decoupling reasoning and knowledge injection for in-context knowledge editing. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24543–24562, 2025.
- [23] Yuchen Wu, Liang Ding, Li Shen, and Dacheng Tao. Robust knowledge editing via explicit reasoning chains for distractor-resilient multi-hop qa. *arXiv preprint arXiv:2509.01468*, 2025.
- [24] Haizhou Shi, Zihao Xu, Hengyi Wang, Weiye Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey. *ACM Computing Surveys*, 58(5):1–42, 2025.
- [25] James L McClelland, Bruce L McNaughton, and Randall C O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419, 1995.
- [26] Nicole Hsing. MIRROR: Complementary encoding and reconstructive consolidation for persistent state in LLM systems. In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*, 2026. URL <https://openreview.net/forum?id=Ivi04bIZc7>.
- [27] Qianqian Shi, Faqiang Liu, Hongyi Li, Guangyu Li, Luping Shi, and Rong Zhao. Hybrid neural networks for continual learning inspired by corticohippocampal circuits. *Nature Communications*, 16(1):1272, 2025.
- [28] Anthony Bazhenov, Pahan Dewasurendra, Giri P Krishnan, and Jean Erik Delanois. Unsupervised replay strategies for continual learning with limited data. In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–10. IEEE, 2024.
- [29] Omer Levy, Minjoon Seo, Eunsol Choi, and Luke Zettlemoyer. Zero-shot relation extraction via reading comprehension. In *Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL 2017)*, pages 333–342, Vancouver, Canada, 2017. Association for Computational Linguistics. doi: 10.18653/v1/K17-1034.
- [30] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

A Extended dataset and benchmark details

This section details the mathematical formulations, empirical distributions, and algorithmic synthesis of the streaming benchmark introduced in Section 4.1.

A.1 Offline routing dataset and behavioral supervision

We build on the Zero-Shot Relation Extraction (ZsRE) benchmark [29], which pairs each factual (*subject, relation, object*) triple with crowd-sourced paraphrases of the underlying relation. After enforcing a strict “exactly $m=30$ paraphrased probes per fact” filter, we retain $N=201,352$ unique fact instances f_i , each accompanied by a set of factual probes $\mathcal{Q}_i = \{(q_{ij}, a_{ij})\}_{j=1}^m$. The queries (q_{ij}) within these m diverse probes evaluate the model’s robustness to linguistic variations of the same underlying knowledge, yielding a total of $Nm=6,040,560$ distinct probes.

To establish behavioral labels that serve as evaluation targets for memory admission, analytical markers for tracking parametric memory shifts, and supervision for classifier baselines, we evaluate a frozen baseline LLM (qwen3-8B, greedy decoding) twice per probe: once *zero-shot* (using only the query q_{ij}) and once *memory-supported* (where the corresponding fact f_i is prepended to q_{ij}). Comparing the two generated responses against the gold answer a_{ij} partitions the probes into four operational classes: *internal-answerable* (correct zero-shot), *missing-knowledge* (refused zero-shot but correct with the fact; corresponding to m_{new}), *stale-knowledge* (incorrect zero-shot but correct with the fact; corresponding to m_{update}), and *unsolved* (incorrect even when the fact f_i is provided). The substantial proportion of *stale-knowledge* cases (19.4% of probes) is particularly informative: it identifies a large slice of parametric knowledge that is not merely missing but *stale*, which an effective memory must *override* rather than simply supplement.

To aggregate these probe-level signals into a fact-level taxonomy, we compute the overall fraction of probes requiring external memory support (ρ_i) and the fraction of these memory-dependent probes indicating stale parametric memory (γ_i):

$$\rho_i = \frac{m_{\text{new}} + m_{\text{update}}}{m}, \quad \gamma_i = \frac{m_{\text{update}}}{m_{\text{new}} + m_{\text{update}}} \quad (9)$$

Based on these metric ratios, the fact-level write label ℓ_i is strictly determined as follows: *non-write* if $\rho_i < 0.01$; *write-update* if $\rho_i \geq 0.01$ and $\gamma_i \geq 0.5$; and *write-new* otherwise ($\rho_i \geq 0.01$ and $\gamma_i < 0.5$). This deterministic assignment provides the ground-truth labels for training and evaluating the memory admission policies. The empirical class proportions are *non-write:write-new:write-update* = 23.5:45.1:31.4 (%). The inclusion of a non-trivial *non-write* fraction is essential to penalize indiscriminate memory admission.

A.2 Streaming evaluation protocol and episode synthesis

We instantiate $E=300$ test-split *episodes*, each modeled as a finite-horizon environment of $T=500$ turns over a private fact pool of size $K=100$. Facts are drawn without replacement from the test split to ensure no fact appears in more than one episode. A typical interaction trajectory is formally structured as follows:

Episodes are synthesized automatically by a scheduler that samples from the fact pool subject to four critical properties:

- (P1) Coverage.** Every sampled fact is injected exactly once and probed at least once.
- (P2) Causality.** No query q_{kj} testing f_k may appear before the turn at which f_k is injected.
- (P3) Liveness.** Turn 0 admits at least one knowledge injection, ensuring the first query is causally answerable.
- (P4) Distractor exposure.** The empirical class mix matches the underlying test-split distribution, exposing the agent to *non-write* distractors at their natural base rate.

Algorithm design and guarantees. To satisfy these temporal and causal constraints efficiently, we employ a three-stage deterministic generation pipeline (Algorithm 1) driven by a seeded random number generator.

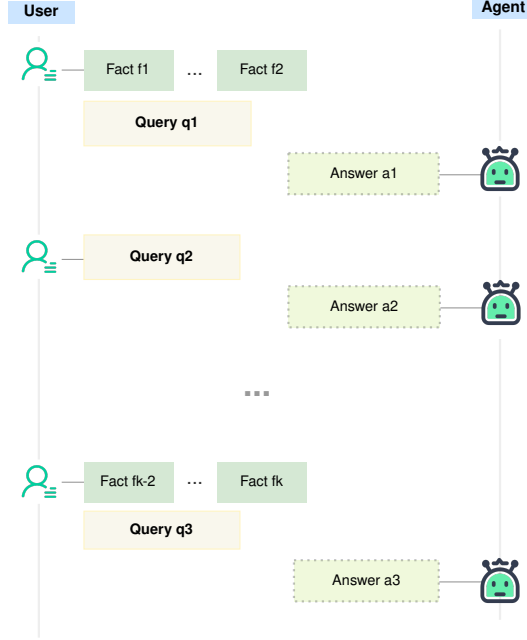


Figure 3: Illustration of the streaming interaction format used in our benchmark.

First, facts are distributed across the timeline using a stratified sampling approach (**Stage 1**). Instead of randomly assigning injection times, the timeline is partitioned into buckets, guaranteeing that every fact is injected early enough to leave sufficient room for subsequent probes. A post-hoc check ensures at least one fact is injected at turn 0 to satisfy liveness (P3).

Second, the algorithm allocates a precise probe budget to each fact (**Stage 2**). It uses a forward greedy pass to dynamically assign probe counts, ensuring the total number of queries perfectly matches the episode length (T). Because of the stratified injection in Stage 1, the algorithm inherently avoids bottlenecks where too many facts are injected too late to be probed.

Finally, probes are sequentially streamed to the agent (**Stage 3**). We maintain an active pool of valid queries ($\mathcal{P}$). A query only enters this active pool at or after the exact turn its corresponding fact is injected. By strictly sampling queries from $\mathcal{P}$, the algorithm naturally enforces causal admissibility (P2).

This pipeline runs in $O(T \log T)$ time and provides a highly robust, bit-exactly reproducible mechanism. By relying on structural scheduling rather than complex constraint solvers, it seamlessly simulates continuous knowledge evolution while strictly respecting all evaluation requirements.

A.3 Evaluation metrics

We report three families of metrics. Downstream QA quality is scored per turn by evaluating the agent’s generated response to the current query against the corresponding ground-truth answer using Exact Match (EM), token-level F_1 , and refusal rate. Storage decision quality is scored at the knowledge-injection level against the fact-level labels ℓ_i via storage ratio together with Store Precision, Store Recall, and Store F_1 . Here, behavioral labels *write-new* and *write-update* are merged and treated as positive targets for memory admission, while *non-write* acts as the negative target. Finally, system efficiency is reported as routing compute cost (inference overhead per decision).

B Implementation of baselines and compared methods

Due to space constraints in the main text, we detail the implementation of heuristic and supervised baselines here:

Algorithm 1 Episode synthesis (K, T, m, rng) .

Stage 1: Stratified injection schedule

- 1: **for** $k = 1, \dots, K$ **do**
- 2: $l_k \leftarrow \lfloor (k-1)T/K \rfloor$, $r_k \leftarrow \min\{T-1, \lfloor kT/K \rfloor - 1\}$
- 3: $\iota(k) \sim \mathcal{U}\{l_k, \dots, r_k\}$ $\triangleright$ Assign fact injection turns $\iota(k)$
- 4: **end for**
- 5: **if** $0 \notin \iota([K])$ **then** $\iota(\arg \min_k \iota(k)) \leftarrow 0$
- 6: **end if**

Stage 2: Probe budget allocation

- 7: $\sigma \leftarrow$ permutation of $[K]$ in ascending ι ; $n \leftarrow \mathbf{1}_K$; $c \leftarrow K$
- 8: **for** $r = 1, \dots, K$ **do**
- 9: $t^* \leftarrow \iota(\sigma(r+1))$ if $r < K$ else T ; $s \leftarrow t^* - c$
- 10: **advance** $n_{\sigma(r)}, n_{\sigma(r-1)}, \dots$ in turn (up to cap m) until s is absorbed, updating c
- 11: **end for**
- 12: Distribute the residual $T - \sum_k n_k$ uniformly at random subject to $n_k \leq m$

Stage 3: Causal probe streaming

- 13: **for** $k = 1, \dots, K$ **do** $P_k \leftarrow$ uniform sample of n_k probes from $\mathcal{Q}_k$
- 14: **end for**
- 15: $\mathcal{P} \leftarrow \emptyset$
- 16: **for** $t = 0, \dots, T-1$ **do**
- 17: $\mathcal{P} \leftarrow \mathcal{P} \cup \{(k, p) : p \in P_k, \iota(k)=t\}$ $\triangleright$ Add probe to pool only after injection
- 18: $\pi(t) \leftarrow$ uniform sample from $\mathcal{P}$; $\mathcal{P} \leftarrow \mathcal{P} \setminus \{\pi(t)\}$
- 19: **end for**
- 20: **return** (ι, n, π)

- **Heuristic Store:** Explicitly probes the model before making a write decision. Given an incoming fact, it generates a factual question, answers that question, and uses a high/low confidence judgment to decide whether the fact should be stored.
- **PPL-Conditional Store:** Thresholds the model’s per-token cross-entropy on the input fact, using linguistic surprisal as a proxy for prior familiarity. The threshold is selected via a validation split.
- **Logistic Regression & MLP Classifier:** Trained on automatically constructed binary *write* vs. *non-write* labels. The MLP Classifier shares the exact same architecture as our large router M_{large} and uses the same semantic/uncertainty features, but is optimized with standard binary cross-entropy (BCE) instead of our reward-based objective.

C Training and deployment protocols

Training proceeds stagewise. We first train the small router and the large router to regress reward targets using mean squared error. After freezing both routers, we construct gate supervision from their greedy actions and train the gate to predict escalation gain. In deployment, routing policies are not chosen by maximizing QA Exact Match (EM) alone, since that would trivially collapse toward writing everything. Instead, we sweep (λ_s, λ_e) on the validation split, tracing the Pareto frontier in the EM-storage-compute space to select the optimal deployment trade-off.

D Extended router architectural ablations

We analyzed the write router in the controlled offline setting to isolate the contributions of feature design, pooling strategy, loss function, and cascade gating design.

D.1 Feature ablation

We performed a 3×3 ablation study comparing feature sources from M_{small} , M_{large} , and their concatenation against semantic, uncertainty, and combined features.

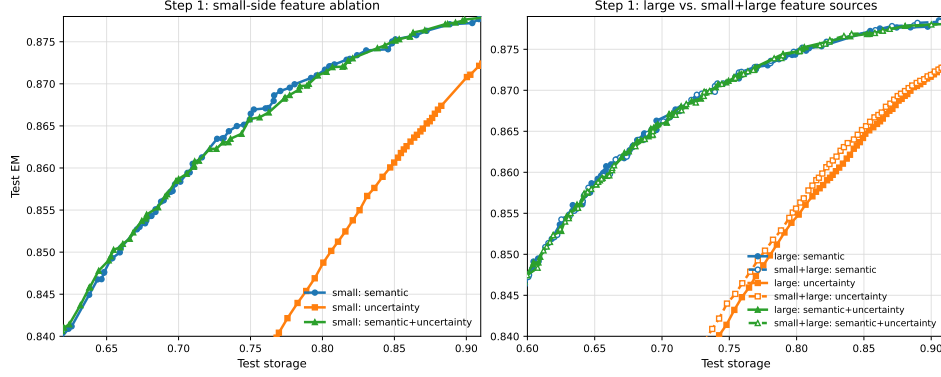


Figure 4: Step 1 single-layer feature ablation on the held-out test set.

As shown in Figure 4, uncertainty-only inputs are clearly weaker than semantic inputs on the small side: to reach test EM 0.86, `single_small_uncertainty` requires storage 0.847, whereas `single_small_semantic` requires only 0.710. On the large router, concatenating small-side features offers almost no benefit: at storage ≤ 0.8 , `single_large_semantic` reaches test EM 0.874 versus 0.875 for `single_small_large_semantic`. At EM ≥ 0.86 , the required storage is 0.658 versus 0.662.

D.2 Pooling and loss ablation

Retaining the strongest feature candidates, we compared pooling strategies (mean vs. `last_token`) and training objectives (MSE vs. Margin Ranking).

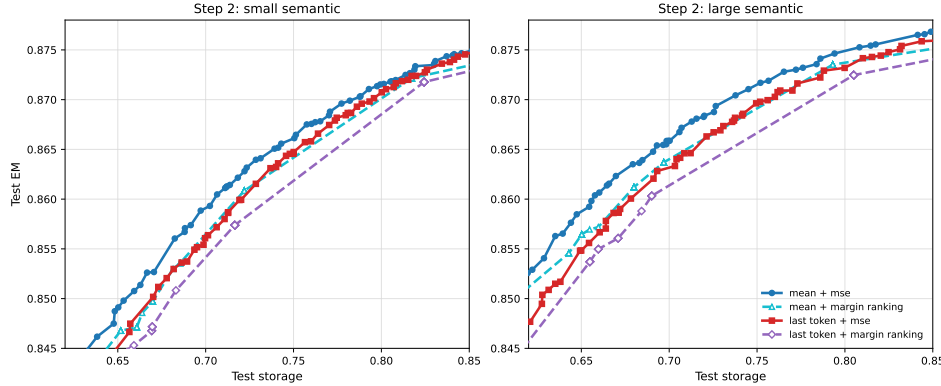


Figure 5: Step 2 pooling-and-loss ablation on the held-out test set.

Figure 5 shows mean pooling with MSE gives the strongest overall frontier. On the large-semantic router, at storage ≤ 0.8 , `mean+mse` reaches test EM 0.875, compared with 0.874 for `mean+margin_ranking`, 0.873 for `last_token+mse`, and 0.860 for `last_token+margin_ranking`. At EM ≥ 0.86 , it requires storage 0.658, versus 0.680, 0.678, and 0.690, respectively.

D.3 Escalation thresholds analysis

In Section 4.4, we summarized the thresholded cascade’s performance. Here we provide the precise ablation numbers supporting Figure 2. At storage caps 0.6, 0.7, and 0.8, the $\rho \leq 0.6$ cascade frontier is only 0.0004, 0.0022, and 0.0010 EM below the large router frontier, **while improving over the small router frontier by 0.0120, 0.0048, and 0.0021, respectively**. Even under the stricter $\rho \leq 0.4$ budget, the gap to the large router stays within 0.0031 over the same range, proving that moderate escalation tightly bounds the theoretical optimum.

D.4 Cascade gate design ablation

To isolate the contribution of the escalation controller itself, we conducted a dedicated cascade-gate ablation.

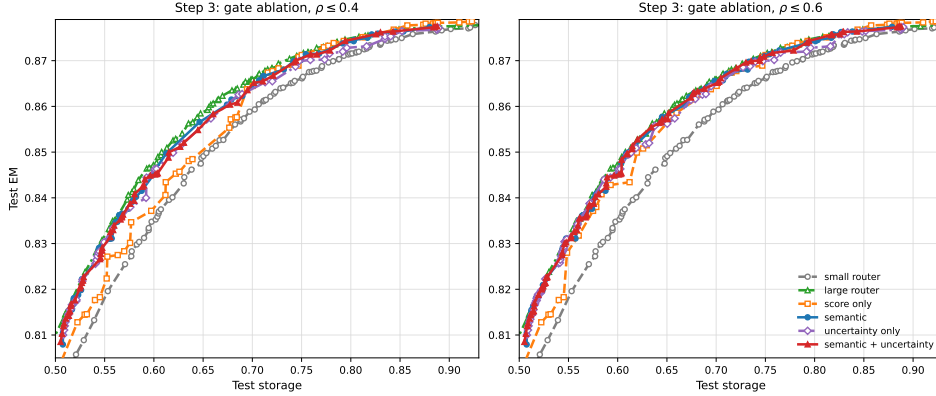


Figure 6: Step 3 cascade-gate ablation on the held-out test set under escalation thresholds $\rho \in \{0.4, 0.6\}$.

Figure 6 shows that all learned gates close most of the gap between the small router and large router anchors. Under $\rho \leq 0.6$, the `cascade_gate_semantic` gate reaches test EM 0.8451 at storage ≤ 0.6 and 0.8656 at storage ≤ 0.7 , compared with 0.8428 and 0.8638 for `cascade_gate_score_only`. The full `cascade_gate_semantic_uncertainty` gate then recovers the strongest high-storage endpoint, reaching EM 0.8743 at storage ≤ 0.8 , only 0.0003 below the large router frontier. Under the stricter $\rho \leq 0.4$ budget, `cascade_gate_semantic_uncertainty` also reaches EM ≥ 0.85 at the lowest storage (0.6267), compared with 0.6464 for `cascade_gate_semantic` and 0.6766 for `cascade_gate_score_only`.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The main claims regarding the dual-layer memory architecture, cost-aware write routing, and slow consolidation are clearly stated in Section 1 and directly supported by the empirical evaluations in Section 4.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The limitations, including the computational cost of offline SFT write-back and the lack of a principled mechanism to resolve temporally conflicting updates, are explicitly discussed in Section 5.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is primarily empirical and systems-oriented; it does not present formal theoretical proofs or theorems.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The full experimental setup, baseline definitions, hyperparameter trade-offs, routing methodology, and the deterministic dataset synthesis algorithms are detailed in Sections 3 and 4, as well as Appendices A-C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: As stated in the abstract, the code and synthesized dataset will be fully released upon acceptance. Detailed algorithmic descriptions and implementation protocols are provided in Section 3, Section 4, and Appendix A-C to facilitate reproduction.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The training strategies, data splits, and evaluation metrics are detailed in Section 4 and Appendix A. The hyperparameter selection process (e.g., sweeping penalty terms λ_s , λ_e on the validation split) and specific optimization details are explicitly documented in Appendix B and C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Error bars are not reported because the online streaming benchmark evaluates deterministic routing policies over a fixed, large-scale dataset of 300 test episodes (150,000 queries total), making variance negligible.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We explicitly state the hardware compute resources (a node equipped with 4 NVIDIA H20 GPUs) used for the experiments in Section 4.2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research is strictly methodological and fully conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper focuses on foundational agent memory optimization and architecture design. It does not introduce specific downstream societal risks beyond those inherent to general large language models.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We do not release any new pre-trained base models or scraped datasets that pose a high risk for misuse.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing assets, including the foundational ZsRE benchmark and the Qwen base models used for evaluation, are appropriately cited in the main text.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper introduces multiple new assets: (1) A novel streaming memory benchmark, whose synthesis algorithm (Algorithm 1), behavioral labeling taxonomy, and structural guarantees are thoroughly documented in Appendix A. (2) The codebase and algorithmic framework for the Dual-Layer Agentic Memory, which includes the implementations of the small-to-large cost-aware write router cascade, the escalation gating mechanism, and the periodic write-back (SFT) pipeline. Detailed mathematical formulations, training objectives, and deployment protocols for these algorithms are provided in Section 3 and Appendices B and C. As stated in the abstract, the complete benchmark datasets, evaluation environment, and framework source code will be publicly released upon acceptance to ensure full reproducibility.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The research does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The research does not involve human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs were used solely for writing assistance and text editing (e.g., grammar, spelling, word choice), which does not impact the core methodology and does not require a formal declaration per NeurIPS guidelines.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.